PYTHON ACTIVITY-BASED LEARNING

Complete Solutions for 10 Case Studies

Beginner-friendly Python programs using the assigned syllabus

Prepared from	Python Activity-Based Learning case studies PDF
Selected topics	Conditions, loops, strings, lists, nested lists, tuples, operators
Included for each case	Problem, objectives, assumptions, algorithm, flowchart, pseudocode, code, tests, output, conclusio

Note: Thresholds, rates, and sample policies in this document are explicit classroom assumptions where the assignment sheet does not specify numerical values. Replace them with your faculty's exact rules if provided.

How to Use This Document

The PDF assignment asks students to select any 10 case studies and prepare the listed deliverables. This document provides one beginner-friendly solution approach for each selected case. Read the assumptions before running a program; the program follows those assumptions.

For the flowcharts, text in square brackets represents a process and text in braces represents a decision. Arrows show the direction of control flow. You can redraw these as standard flowchart symbols if your faculty requires drawn diagrams.

Selected case studies: 1, 7, 11, 12, 17, 23, 29, 42, 57, and 66 from the supplied PDF.

1. University Scholarship Allocation

Problem Statement

Create a program that checks scholarship applicants using aggregate marks, attendance, and family income. Eligible applicants are ranked by marks; ties are resolved using higher attendance.

Objectives

• Check each applicant against minimum eligibility rules.
• Rank eligible applicants using marks and attendance.
• Display eligible and ineligible applicants clearly.

Assumptions

• Minimum aggregate marks: 75%.
• Minimum attendance: 85%.
• Maximum annual family income: ■250,000.
• Each applicant is entered with name, marks, attendance, and income.

Algorithm

1. Start and read the number of applicants.
2. For each applicant, read the four details.
3. Check all three eligibility conditions.
4. Store eligible applicants as tuples (name, marks, attendance).
5. Sort eligible applicants by marks, then attendance, both in descending order.
6. Display the ranked list and end.

Flowchart

[START] → [Read applicant count] → [Read each applicant] → {Marks ≥ 75, attendance ≥ 85, income ≤ 250000

Pseudocode

```
START\nINPUT n\neligible = empty list\nREPEAT n times:\n    INPUT name, marks, attendance, income\n
   IF marks >= 75 AND attendance >= 85 AND income <= 250000:\n        ADD (name, marks, attendance)
to eligible\nSORT eligible by marks descending, then attendance descending\nDISPLAY eligible list
with ranks\nSTOP
```

Python Source Code

```
n = int(input("Number of applicants: "))
eligible = []

for i in range(n):
    print("\\nApplicant", i + 1)
    name = input("Name: ")
    marks = float(input("Aggregate marks (%): "))
    attendance = float(input("Attendance (%): "))
    income = float(input("Annual family income (■): "))

    if marks >= 75 and attendance >= 85 and income <= 250000:
        eligible.append((name, marks, attendance))
    else:
        print(name, "is not eligible.")

eligible.sort(key=lambda item: (item[1], item[2]), reverse=True)

print("\\nScholarship shortlist:")
if len(eligible) == 0:
    print("No eligible applicants.")
else:
    for rank, applicant in enumerate(eligible, start=1):
```

```
        print(rank, applicant[0], "- Marks:", applicant[1],
              "Attendance:", applicant[2])
```

Test Cases

Input / scenario	Expected result
Asha	82
Ravi	74
Meena	82

Sample Output

```
Number of applicants: 2
Applicant 1: Asha, 82, 90, 200000
Applicant 2: Ravi, 74, 95, 100000
Scholarship shortlist:
1 Asha - Marks: 82.0 Attendance: 90.0
```

Conclusion

The program applies the stated scholarship rules and ranks eligible applicants consistently. The thresholds are assumptions for this activity and can be changed to match a supplied policy.

2. Retail Billing with Multiple Discount Rules

Problem Statement

Prepare an itemized bill for products using quantity, unit price, a bill-level discount for purchases above a threshold, and category-specific tax rates.

Objectives

• Calculate each item's subtotal and the total purchase value.
• Apply a discount when the subtotal reaches the threshold.
• Calculate tax and display the final payable amount.

Assumptions

• A 10% discount is applied when subtotal is at least ■2,000.
• Discount is applied before tax.
• Tax rates: groceries 5%, electronics 18%, clothing 12%.
• Only the categories listed above are accepted.

Algorithm

1. Read the number of items.
2. For each item, read name, category, price, and quantity.
3. Calculate line total and add it to subtotal.
4. Apply the threshold discount if eligible.
5. Calculate category-wise tax on the discounted amount.
6. Display subtotal, discount, tax, and final total.

Flowchart

[START] → [Read items] → [Calculate line totals and subtotal]\n → {Subtotal ≥ ■2000?} → [Apply

Pseudocode

```
INPUT n; subtotal = 0; items = empty list\nREPEAT n times: INPUT name, category, price, quantity\n
  line_total = price * quantity\n    ADD item details and line_total to items\n    subtotal =
subtotal + line_total\nIF subtotal >= 2000: discount = subtotal * 0.10\nELSE: discount = 0\
ndiscounted_total = subtotal - discount\nFOR each item: tax = discounted line amount * category tax
rate\nDISPLAY subtotal, discount, tax, and final total
```

Python Source Code

```
n = int(input("Number of different items: "))
items = []
subtotal = 0

rates = {"groceries": 0.05, "electronics": 0.18, "clothing": 0.12}

for i in range(n):
    print("\\nItem", i + 1)
    name = input("Product name: ")
    category = input("Category (groceries/electronics/clothing): ").lower()
    price = float(input("Unit price: "))
    quantity = int(input("Quantity: "))

    if category not in rates or price < 0 or quantity <= 0:
        print("Invalid item; please restart with valid details.")
        break

    line_total = price * quantity
    subtotal += line_total
    items.append((name, category, line_total))
else:
    discount = subtotal * 0.10 if subtotal >= 2000 else 0
    discounted_total = subtotal - discount
    tax = 0

    print("\\nITEMIZED BILL")
    for item in items:
        share = item[2] / subtotal if subtotal > 0 else 0
        discounted_line = share * discounted_total
        item_tax = discounted_line * rates[item[1]]
        tax += item_tax
        print(item[0], ":", round(item[2], 2),
              "Tax:", round(item_tax, 2))

    print("Subtotal: ■", round(subtotal, 2))
    print("Discount: ■", round(discount, 2))
    print("Tax: ■", round(tax, 2))
    print("Final total: ■", round(discounted_total + tax, 2))
```

Test Cases

Input / scenario	Expected result
Notebook, groceries, ■100, quantity 2	Subtotal ■200; no discount; tax ■10
Headphones, electronics, ■2500, quantity 1	Subtotal ■2500; discount ■250; tax on discounted amount ■405; final ■2655

Sample Output

```
Subtotal: ■ 2500.0
Discount: ■ 250.0
Tax: ■ 405.0
Final total: ■ 2655.0
```

Conclusion

The program demonstrates lists, tuples, dictionaries, loops, conditions, and arithmetic for billing. Discount and tax rules are illustrative assumptions.

3. Examination Result and Grace-Mark Validation

Problem Statement

Calculate a student's final result across subjects using subject-wise pass marks and a limited grace-mark policy without allowing marks to exceed the maximum.

Objectives

• Read marks for multiple subjects.
• Apply a small grace allowance only when the policy permits it.
• Display final marks and pass/fail status for each subject.

Assumptions

• Each subject is out of 100 and requires at least 40 to pass.
• At most 5 grace marks may be added to one subject that has 35–39 marks.
• Grace marks cannot raise a subject above 100.
• The student passes overall only if every subject is passed after permitted grace marks.

Algorithm

1. Read the number of subjects and each subject's marks.
2. Validate that every mark is between 0 and 100.
3. If marks are 35–39, add only enough grace marks to reach 40, provided the total grace used is at most 5.
4. Keep each final mark at or below 100.
5. Display each subject result and overall status.

Flowchart

[START] → [Read subject marks] → {Marks in 0..100?}\n NO → [Report invalid input] YES → {Marks 35..

Pseudocode

```
INPUT number of subjects\ngrace_used = 0; all_pass = True\nFOR each subject:\n    INPUT marks\n
IF marks < 0 OR marks > 100: report invalid input\n    ELSE IF 35 <= marks < 40 AND grace_used + (
40 - marks) <= 5:\n        add 40 - marks as grace; marks = 40\n    IF marks < 40: all_pass = False\
n    DISPLAY final marks and status\nDISPLAY overall result
```

Python Source Code

```
n = int(input("Number of subjects: "))
grace_used = 0
all_pass = True

for i in range(n):
    marks = int(input("Marks for subject " + str(i + 1) + " (0-100): "))

    if marks < 0 or marks > 100:
        print("Invalid marks. Enter marks from 0 to 100.")
        all_pass = False
        continue

    original = marks
    if 35 <= marks < 40:
        needed = 40 - marks
        if grace_used + needed <= 5:
            marks += needed
            grace_used += needed

    if marks >= 40:
        print("Subject", i + 1, ":", original, "->", marks, "- PASS")
    else:
        print("Subject", i + 1, ":", original, "->", marks, "- FAIL")
        all_pass = False

print("Grace marks used:", grace_used)
if all_pass:
    print("Overall result: PASS")
else:
    print("Overall result: FAIL")
```

Test Cases

Input / scenario	Expected result
Marks 38, 65	38 becomes 40 using 2 grace marks; both subjects pass

Input / scenario	Expected result
Marks 36, 37	Needs 4 + 3 grace marks; 5-mark total limit means one subject remains failed

Sample Output

```
Marks: 38, 65
Subject 1 : 38 -> 40 - PASS
Subject 2 : 65 -> 65 - PASS
Grace marks used: 2
Overall result: PASS
```

Conclusion

The program applies the example grace policy while preserving the 100-mark maximum. Actual institutional policies should replace the assumptions before real use.

4. Attendance Recovery Computation

Problem Statement

Calculate current attendance percentage and find the minimum number of consecutive classes a student must attend to reach a required attendance threshold.

Objectives

• Calculate attendance percentage from attended and conducted classes.
• Estimate how many future classes must be attended consecutively.
• Show the result for the chosen eligibility threshold.

Assumptions

• The threshold is 75%.
• The student attends every future class during the recovery period.
• Attended classes cannot exceed conducted classes; counts cannot be negative.

Algorithm

1. Read attended classes, total classes, and required percentage.
2. Validate the input counts.
3. Calculate current attendance percentage.
4. If the student already meets the threshold, display zero required classes.
5. Otherwise add one future attended class at a time until the threshold is met.

Flowchart

[START] → [Read counts and threshold] → {Inputs valid?}\n → [Calculate percentage] → {Threshold reached

Pseudocode

```
INPUT attended, total, threshold\nIF counts invalid: DISPLAY error\nELSE:\n    current = attended /
total * 100 (if total > 0)\n    extra = 0\n    WHILE (attended + extra) / (total + extra) * 100 <
threshold:\n        extra = extra + 1\n    DISPLAY current and extra
```

Python Source Code

```
attended = int(input("Classes attended: "))
total = int(input("Total classes conducted: "))
threshold = float(input("Required attendance percentage: "))

if attended < 0 or total < 0 or attended > total:
    print("Invalid attendance counts.")
elif threshold <= 0 or threshold > 100:
    print("Threshold must be greater than 0 and at most 100.")
else:
    current = attended / total * 100 if total > 0 else 0
    extra = 0

    while (attended + extra) / (total + extra) * 100 < threshold:
        extra += 1

    print("Current attendance:", round(current, 2), "%")
    print("Consecutive classes to attend:", extra)
    print("Attendance after recovery:",
          round((attended + extra) / (total + extra) * 100, 2), "%")
```

Test Cases

Input / scenario	Expected result
Attended 60, total 100, threshold 75	Current 60%; attend 60 consecutive classes to reach 75%
Attended 80, total 100, threshold 75	Current 80%; 0 additional classes

Sample Output

```
Classes attended: 60
Total classes conducted: 100
Required attendance percentage: 75
Current attendance: 60.0 %
Consecutive classes to attend: 60
Attendance after recovery: 75.0 %
```

Conclusion

The loop finds the smallest number of future classes needed under the assumption that every future class is attended.

5. Student Performance Trend Analysis

Problem Statement

Compare a student's scores in successive examinations, report improvements or declines, and identify subjects that remain weak.

Objectives

• Store subject names and scores in lists.
• Calculate score changes between two examinations.
• Identify subjects below a chosen score threshold.

Assumptions

• Scores are percentages from 0 to 100.
• The two examinations contain the same subjects in the same order.
• A score below 40% is classified as weak.

Algorithm

1. Read the subject names.
2. Read the earlier and later score for each subject.
3. Validate scores are between 0 and 100.
4. Calculate the difference between later and earlier scores.
5. Classify each subject as improved, declined, or unchanged and flag weak subjects.

Flowchart

[START] → **[Read subjects and two scores]** → **[Validate scores]\n** → **[Compute later - earlier]** → **[Classify**

Pseudocode

```
INPUT n\nFOR each subject: INPUT name, old_score, new_score\n    change = new_score - old_score\n
 IF change > 0: trend = Improved\n    ELSE IF change < 0: trend = Declined\n    ELSE: trend =
Unchanged\n    IF new_score < 40: mark as weak\n    DISPLAY subject, change, trend, weak status
```

Python Source Code

```
n = int(input("Number of subjects: "))
subjects = []
old_scores = []
new_scores = []

for i in range(n):
    name = input("Subject name: ")
    old = float(input("Earlier exam score (%): "))
    new = float(input("Later exam score (%): "))

    if old < 0 or old > 100 or new < 0 or new > 100:
        print("Invalid score. Use values from 0 to 100.")
        continue

    subjects.append(name)
    old_scores.append(old)
    new_scores.append(new)

print("\\nPERFORMANCE REPORT")
for i in range(len(subjects)):
    change = new_scores[i] - old_scores[i]

    if change > 0:
        trend = "Improved"
    elif change < 0:
        trend = "Declined"
    else:
        trend = "Unchanged"

    weak = "YES" if new_scores[i] < 40 else "NO"
    print(subjects[i], "| Change:", round(change, 2),
          "| Trend:", trend, "| Weak subject:", weak)
```

Test Cases

Input / scenario	Expected result
Python 55→70	Change +15; Improved; not weak

Input / scenario	Expected result
Maths 45→35	Change -10; Declined; weak subject

Sample Output

```
PERFORMANCE REPORT
Python | Change: 15.0 | Trend: Improved | Weak subject: NO
Maths | Change: -10.0 | Trend: Declined | Weak subject: YES
```

Conclusion

The program uses lists, loops, conditions, and arithmetic to show trends and highlight subjects that may need more practice.

6. Student Timetable Conflict Detection

Problem Statement

Compare class periods stored as tuples and detect whether two classes scheduled on the same day overlap in time.

Objectives

• Represent each class using a tuple.
• Compare day and time intervals.
• Report overlapping classes.

Assumptions

• Times are entered as 24-hour integer minutes from midnight.
• Each class has a start time earlier than its end time.
• Two periods overlap if they share any time; one ending exactly when another starts is not a conflict.

Algorithm

1. Read the number of classes.
2. For each class, read course name, day, start time, and end time.
3. Store each class as a tuple.
4. Compare each pair of classes.
5. If they are on the same day and their intervals overlap, display a conflict.

Flowchart

[START] → [Read class tuples] → [Compare each pair]\n → {Same day AND start1 < end2 AND start2 < end1?}

Pseudocode

```
INPUT n; classes = empty list\nREPEAT n times: INPUT course, day, start, end; ADD tuple\nFOR i from
0 to n-1:\n    FOR j from i+1 to n-1:\n        IF day[i] == day[j] AND start[i] < end[j] AND start[
j] < end[i]:\n            DISPLAY conflict between course[i] and course[j]
```

Python Source Code

```
n = int(input("Number of classes: "))
classes = []

for i in range(n):
    print("Class", i + 1)
    course = input("Course name: ")
    day = input("Day: ").lower()
    start = int(input("Start time in minutes after midnight: "))
    end = int(input("End time in minutes after midnight: "))

    if start < 0 or end > 1440 or start >= end:
        print("Invalid time interval; class skipped.")
    else:
        classes.append((course, day, start, end))

found = False
for i in range(len(classes)):
    for j in range(i + 1, len(classes)):
        a = classes[i]
        b = classes[j]

        if a[1] == b[1] and a[2] < b[3] and b[2] < a[3]:
            print("Conflict:", a[0], "and", b[0], "on", a[1])
            found = True

if not found:
    print("No timetable conflicts found.")
```

Test Cases

Input / scenario	Expected result
Python Monday 09:00–10:00; Maths Monday 09:30–10:30	Conflict detected
Python Monday 09:00–10:00; Maths Tuesday 09:30–10:30	No conflict

Sample Output

```
Conflict: Python and Maths on monday
```

Conclusion

Tuple records and nested loops make it possible to compare every pair of classes without needing advanced libraries.

7. Password Policy Validation Engine

Problem Statement

Check a sample password against simple rules and explain which rules are not satisfied.

Objectives

• Check password length.
• Check for uppercase, lowercase, digit, and special character.
• Report every failed condition.

Assumptions

• Minimum length is 8 characters.
• A valid password must contain uppercase and lowercase letters, at least one digit, and at least one special character from !@#$%.

Algorithm

1. Read a sample password.
2. Check its length.
3. Use a loop to check each character category.
4. Store failed rules in a list.
5. Display all failed rules or confirm that the sample passes.

Flowchart

[START] → [Read password] → [Check length and character types]\n → {Any rule failed?} YES → [Display fa

Pseudocode

```
INPUT password; errors = empty list\nIF length < 8: ADD length error\nSet uppercase, lowercase,
digit, special flags to False\nFOR each character: update matching flags\nIF any flag is False: ADD
related error\nIF errors empty: DISPLAY valid\nELSE: DISPLAY errors
```

Python Source Code

```
password = input("Enter a sample password: ")
errors = []

if len(password) < 8:
    errors.append("Must contain at least 8 characters.")

upper = False
lower = False
digit = False
special = False

for ch in password:
    if ch.isupper():
        upper = True
    elif ch.islower():
        lower = True
    elif ch.isdigit():
        digit = True
    elif ch in "!@#$%":
        special = True

if not upper:
    errors.append("Add at least one uppercase letter.")
if not lower:
    errors.append("Add at least one lowercase letter.")
if not digit:
    errors.append("Add at least one digit.")
if not special:
    errors.append("Add at least one special character from !@#$%.")

if len(errors) == 0:
    print("Password satisfies the sample policy.")
else:
    print("Password does not satisfy the policy:")
    for error in errors:
        print("-", error)
```

Test Cases

Input / scenario	Expected result
Abcdef1!	Passes all sample rules

Input / scenario	Expected result
abcdefg	Fails length, uppercase, digit, and special-character rules

Sample Output

```
Enter a sample password: abcdefg
Password does not satisfy the policy:
- Must contain at least 8 characters.
- Add at least one uppercase letter.
- Add at least one digit.
- Add at least one special character from !@#$%.
```

Conclusion

The program demonstrates string operations, loops, conditions, and lists. It checks a sample policy only; it does not store passwords or provide real authentication.

8. Student Marks Matrix Analysis

Problem Statement

Store marks of several students across multiple subjects in a nested list, calculate student totals and subject averages, and identify the top performer.

Objectives

• Represent marks as a matrix (nested list).
• Calculate row-wise totals and column-wise averages.
• Find the student with the highest total.

Assumptions

• Each mark is from 0 to 100.
• All students take the same subjects.
• Student names and subject names are unique for this activity.

Algorithm

1. Read student and subject counts.
2. Read subject names and each student's marks.
3. Store each student's marks as one row in a nested list.
4. Calculate each row total and identify the highest total.
5. Calculate each column average and display the report.

Flowchart

[START] → [Read names and marks matrix] → [Calculate row totals]\n → [Find highest total] → [Calculate

Pseudocode

```
INPUT student_count, subject_count\nINPUT subject names\nFOR each student: INPUT name and marks;
store marks row\nFOR each row: calculate total; compare with top total\nFOR each subject column:
add marks and divide by student_count\nDISPLAY totals, top student, and subject averages
```

Python Source Code

```
students = int(input("Number of students: "))
subjects_count = int(input("Number of subjects: "))

subject_names = []
for j in range(subjects_count):
    subject_names.append(input("Subject name: "))

names = []
marks_matrix = []

for i in range(students):
    name = input("Student name: ")
    row = []
    for j in range(subjects_count):
        mark = float(input("Marks in " + subject_names[j] + ": "))
        row.append(mark)
    names.append(name)
    marks_matrix.append(row)

totals = []
for i in range(students):
    total = sum(marks_matrix[i])
    totals.append(total)
    print(names[i], "total:", total)

if students > 0:
    top_index = totals.index(max(totals))
    print("Top performer:", names[top_index])

for j in range(subjects_count):
    column_total = 0
    for i in range(students):
        column_total += marks_matrix[i][j]
    average = column_total / students if students > 0 else 0
    print(subject_names[j], "average:", round(average, 2))
```

Test Cases

Input / scenario	Expected result
Asha: 80, 90; Ravi: 70, 85	Totals: Asha 170, Ravi 155; Asha top; subject averages 75 and 87.5

Sample Output

```
Asha total: 170.0
Ravi total: 155.0
Top performer: Asha
Python average: 75.0
Maths average: 87.5
```

Conclusion

The nested list represents a marks matrix. Nested loops calculate student-wise and subject-wise statistics using basic Python.

9. Smart Canteen Order and Stock Management

Problem Statement

Process customer orders, calculate bills, update available stock, reject orders that exceed stock, and summarize sales.

Objectives

• Store menu prices and stock quantities.
• Validate requested quantities before accepting an order.
• Update stock and calculate total sales.

Assumptions

• The sample menu has three items: idli ■30, dosa ■50, and juice ■25.
• Opening stock is 20 portions of each item.
• Orders use item names in lowercase and positive integer quantities.
• No discounts or taxes are applied.

Algorithm

1. Create menu price and stock lists.
2. Read the number of orders.
3. For each order, read item and quantity.
4. If the item exists, quantity is positive, and stock is sufficient, calculate the bill and reduce stock.
5. Otherwise reject the order.
6. Display total sales and remaining stock.

Flowchart

[START] → [Initialize menu and stock] → [Read order]\n → {Item valid, quantity positive, stock sufficier

Pseudocode

```
prices = menu; stock = initial quantities; sales = 0\nINPUT number of orders\nREPEAT for each order:
\n    INPUT item, quantity\n    IF item exists AND quantity > 0 AND quantity <= stock[item]:\n
  bill = price[item] * quantity\n        stock[item] = stock[item] - quantity\n        sales =
sales + bill\n        DISPLAY bill\n    ELSE: DISPLAY order rejected\nDISPLAY sales and remaining
stock
```

Python Source Code

```
menu = ["idli", "dosa", "juice"]
prices = [30, 50, 25]
stock = [20, 20, 20]
total_sales = 0

orders = int(input("Number of orders: "))

for i in range(orders):
    item = input("Item (idli/dosa/juice): ").lower()
    quantity = int(input("Quantity: "))

    if item in menu and quantity > 0:
        index = menu.index(item)
        if quantity <= stock[index]:
            bill = prices[index] * quantity
            stock[index] -= quantity
            total_sales += bill
            print("Order accepted. Bill: ■", bill)
        else:
            print("Order rejected: insufficient stock.")
    else:
        print("Order rejected: invalid item or quantity.")

print("\\nDaily sales: ■", total_sales)
print("Remaining stock:")
for i in range(len(menu)):
    print(menu[i], ":", stock[i])
```

Test Cases

Input / scenario	Expected result
2 dosa	Accepted; bill ■100; dosa stock becomes 18
25 idli	Rejected because opening stock is 20

Sample Output

```
Order accepted. Bill: ■ 100
Order rejected: insufficient stock.
Daily sales: ■ 100
Remaining stock:
idli : 20
dosa : 18
juice : 20
```

Conclusion

The program uses lists, loops, conditions, and list indexing to track orders and stock. It is a classroom simulation, not a live point-of-sale system.

10. Student Academic Risk Classification

Problem Statement

Use each student's marks, attendance, and laboratory score to classify academic risk and report the reasons for the classification.

Objectives

• Read and store academic indicators for multiple students.
• Identify risk factors using threshold rules.
• Display each student's risk level and reasons.

Assumptions

• High risk: marks below 40, attendance below 65%, or lab score below 40.
• Medium risk: no high-risk condition, but marks below 50, attendance below 75%, or lab score below 50.
• Otherwise the student is classified as low risk.
• Thresholds are examples for this assignment.

Algorithm

1. Read the number of students.
2. For each student, read name, marks, attendance, and lab score.
3. Check each high-risk condition and store its reason.
4. If no high-risk condition exists, check medium-risk conditions.
5. Display the risk category and all identified reasons.

Flowchart

[START] → [Read student data] → [Check high-risk thresholds]\n → {Any high-risk factor?} YES → [HIGH RI

Pseudocode

```
INPUT n\nFOR each student: INPUT name, marks, attendance, lab\n    reasons = empty list\n    IF
marks < 40: add marks reason\n    IF attendance < 65: add attendance reason\n    IF lab < 40: add
lab reason\n    IF reasons not empty: risk = HIGH\n    ELSE:\n        check marks < 50, attendance
< 75, lab < 50\n        IF any medium condition: risk = MEDIUM\n        ELSE: risk = LOW\n
DISPLAY name, risk, reasons
```

Python Source Code

```
n = int(input("Number of students: "))

for i in range(n):
    print("\\nStudent", i + 1)
    name = input("Name: ")
    marks = float(input("Academic marks (%): "))
    attendance = float(input("Attendance (%): "))
    lab = float(input("Laboratory score (%): "))

    reasons = []
    if marks < 40:
        reasons.append("academic marks below 40%")
    if attendance < 65:
        reasons.append("attendance below 65%")
    if lab < 40:
        reasons.append("laboratory score below 40%")

    if len(reasons) > 0:
        risk = "HIGH"
    else:
        if marks < 50:
            reasons.append("academic marks below 50%")
        if attendance < 75:
            reasons.append("attendance below 75%")
        if lab < 50:
            reasons.append("laboratory score below 50%")

        if len(reasons) > 0:
            risk = "MEDIUM"
        else:
            risk = "LOW"

    print("Student:", name)
    print("Risk level:", risk)
    if len(reasons) == 0:
        print("Reasons: No risk factors found.")
    else:
```

```
print("Reasons:")
for reason in reasons:
    print("-", reason)
```

Test Cases

Input / scenario	Expected result
Asha: marks 35, attendance 80, lab 70	HIGH; reason: academic marks below 40%
Ravi: marks 46, attendance 70, lab 48	MEDIUM; reasons: marks below 50%, attendance below 75%, lab below 50%
Meena: 80, 90, 85	LOW; no risk factors

Sample Output

```
Student: Asha
Risk level: HIGH
Reasons:
- academic marks below 40%
```

Conclusion

The program combines multiple indicators and reports transparent reasons for the risk category. The thresholds are illustrative and should be adjusted to the course policy.

Submission Checklist

■ Review the assumptions and replace illustrative thresholds with faculty-provided rules if needed.

■ Run each program in Python and verify it with the included test cases.

■ Capture genuine screenshots of your program and its output from your own Python environment; the sample output in this PDF is text, not a screenshot.

■ If required, redraw the text flowcharts using standard start/end, process, input/output, and decision symbols.

■ Upload the completed materials to your own GitHub repository named Python-Activity-Based-Learning and submit the repository link as instructed in the assignment sheet.

All programs are intended for learning and demonstration. They use basic Python constructs and do not require external packages.